```
import heapq

GOAL_STATE = (1, 2, 3,
              8, 0, 4,
              7, 6, 5)


def misplaced_tiles(state):
    count = 0

    for i in range(9):
        if state[i] != 0 and state[i] != GOAL_STATE[i]:
            count += 1

    return count


def get_neighbors(state):
    neighbors = []

    zero_idx = state.index(0)
    row, col = divmod(zero_idx, 3)

    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dr, dc in directions:
        new_row = row + dr
        new_col = col + dc

        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_idx = new_row * 3 + new_col

            state_list = list(state)

            state_list[zero_idx], state_list[new_idx] = \
                state_list[new_idx], state_list[zero_idx]

            neighbors.append(tuple(state_list))

    return neighbors


def count_inversions(state):
    state_no_zero = [tile for tile in state if tile != 0]
```

```
    inversions = 0

    for i in range(len(state_no_zero)):
        for j in range(i + 1, len(state_no_zero)):
            if state_no_zero[i] > state_no_zero[j]:
                inversions += 1

    return inversions


def is_solvable(start_state):
    start_inv = count_inversions(start_state)
    goal_inv = count_inversions(GOAL_STATE)

    return (start_inv % 2) == (goal_inv % 2)


def a_star_misplaced(start_state):
    start_h = misplaced_tiles(start_state)

    pq = [(start_h, 0, start_state, [start_state])]
    visited = {start_state: 0}

    while pq:
        f, g, current, path = heapq.heappop(pq)

        if current == GOAL_STATE:
            return path

        for neighbor in get_neighbors(current):
            new_g = g + 1

            if neighbor not in visited or new_g < visited[neighbor]:
                visited[neighbor] = new_g

                h = misplaced_tiles(neighbor)
                new_f = new_g + h

                heapq.heappush(
                    pq,
                    (new_f, new_g, neighbor, path + [neighbor])
                )

    return None
```

```
def print_board(state):
    for i in range(0, 9, 3):
        row = [
            str(val) if val != 0 else ' '
            for val in state[i:i + 3]
        ]

        print(" ".join(row))

    print("-" * 9)


def get_user_input():
    print("Enter the 8-puzzle board using numbers 0 to 8.")
    print("Use 0 for the blank space.\n")

    while True:
        try:
            user_input = input("Enter 9 numbers: ").strip().split()

            if len(user_input) != 9:
                print("Enter exactly 9 numbers.")
                continue

            state = tuple(int(x) for x in user_input)

            if sorted(state) != list(range(9)):
                print("Use each number from 0 to 8 exactly once.")
                continue

            return state

        except ValueError:
            print("Enter integers only.")


if __name__ == "__main__":

    initial_state = get_user_input()

    print("\nInitial State:")
    print_board(initial_state)
```

```
print("Goal State:")
print_board(GOAL_STATE)

if not is_solvable(initial_state):
    print("The puzzle is UNSOLVABLE.")
else:
    print("Solving using A* with Misplaced Tiles...\n")

    solution = a_star_misplaced(initial_state)

    if solution:
        print("Solution found!")
        print("Number of steps:", len(solution) - 1)
        print()

        for step, state in enumerate(solution):
            print("Step", step)
            print_board(state)

    else:
        print("No solution found.")
```

```
Enter the 8-puzzle board using numbers 0 to 8.
Use 0 for the blank space.

Enter 9 numbers: 8 1 3 7 2 4 6 0 5


Initial State:
8 1 3
7 2 4
6   5
---------
Goal State:
1 2 3
8   4
7 6 5
---------
Solving using A* with Misplaced Tiles...

Solution found!
Number of steps: 5

Step 0
8 1 3
7 2 4
6   5
---------
Step 1
8 1 3
7 2 4
  6 5
---------
Step 2
8 1 3
  2 4
7 6 5
---------
Step 3
  1 3
8 2 4
7 6 5
---------
Step 4
1   3
8 2 4
7 6 5
---------
Step 5
1 2 3
8   4
7 6 5
```